Casos de Prueba y Reporte de Pruebas (WP.14 y WP.15) — Sprint 2

Estándar: ISO/IEC 29110-5-1-1:2025 (Tablas 26 y 27 - WP.14 y WP.15 | Actividades SI.04 y SI.05)

Nombre del proyecto: SoftGames (Android Studio / Java)

Versión del documento: 2.0 (Avance parcial al 20 de octubre de 2026 — Iteración 2)

Ejecutores de pruebas hasta el corte actual: Rogelio Elian Loera Garay y Sebastian Del Hoyo Alvarado

Cliente: María De León Sigg

1.0 Actividades de Verificación y Validación Realizadas

En atención a las observaciones **c)**, **D)** y **E)** de la auditoría de la Iteración 1, este documento integra tanto la especificación de **Casos y Procedimientos de Prueba (WP.14)** vinculados a los requerimientos y casos de uso del Sprint 2, como el **Reporte de Ejecución de Pruebas y Registro de Defectos (WP.15)** junto con el cálculo formal de la **Densidad de Defectos**.

- **Entorno de pruebas:** Android Studio Emulator (Pixel / API 34) y pruebas de almacenamiento interno en el directorio de archivos de la aplicación.
- **Alcance evaluado al corte del 20 de octubre de 2026:** Se ejecutaron al 100% las pruebas unitarias y de integración correspondientes a los 2 casos de uso ya construidos (**CU-01: Iniciar Modo Maratón** y **CU-02: Registrar y consultar estadísticas de progreso por categoría**), participando **Sebastian Del Hoyo Alvarado** en las pruebas unitarias y revisión de **CU-01** y **Rogelio Elian Loera Garay** en las pruebas de **CU-02** y las pruebas de integración de ambos módulos. Los casos de prueba para **CU-03** y **CU-04** quedan diseñados y pendientes de ejecución al concluir su codificación antes del 29 de octubre.

2.0 Matriz de Casos y Procedimientos de Prueba (WP.14)

ID Prueba	Caso de Uso / Flujo Verificado	Precondiciones y Pasos del Procedimiento	Resultado Esperado	Resultado Obtenido y Fecha	Ejecutor
CP-S2-01	CU-01: Iniciar Modo Maratón (Flujo normal - Selección de categorías y Modo General)	1. Iniciar sesión en una partida activa. 2. Entrar a Modo Maratón. 3. Marcar categorías de ingeniería de software (o activar Modo General) y oprimir "Iniciar Maratón".	El sistema inicia la partida con contador de errores en 0 (3 vidas), racha en 0 y preguntas de las categorías elegidas con dificultad progresiva.	**Aprobado (Pass)** 20/oct/2026	Sebastian Del Hoyo Alvarado
CP-S2-02	CU-01: Iniciar Modo Maratón (Flujo	1. Entrar a la pantalla de configuración de Modo Maratón.	El sistema impide iniciar la partida y muestra una alerta	**Aprobado (Pass)** 20/oct/2026	Sebastian Del Hoyo Alvarado

	alternativo 2.1 - Sin categorías marcadas)	2. Desmarcar todas las categorías y el Modo General. 3. Oprimir "Iniciar Maratón".	indicando que debe seleccionarse al menos una categoría o activar el Modo General.		
CP-S2-03	CU-01: Iniciar Modo Maratón (Flujo normal pasos 7-9 y Flujo alt. 5.1 - Límite de 3 errores y resumen por categoría)	1. Iniciar partida en Modo Maratón. 2. Responder correctamente 4 preguntas para verificar aumento de dificultad. 3. Fallar o dejar agotar el tiempo en 3 preguntas.	Al tercer error finaliza la partida y se muestra la pantalla de resumen con puntaje total, racha alcanzada y los puntos y problemas resueltos por cada categoría seleccionada.	**Aprobado (Pass)** 20/oct/2026 (Tras corregir DEF-S2-01)	Rogelio Elian Loera Garay
CP-S2-04	CU-02: Registrar y consultar estadísticas por categoría (Flujo normal)	1. Completar una partida obteniendo un nuevo puntaje. 2. Abrir la pantalla de "Estadísticas" desde el resumen o el menú principal.	El sistema serializa automáticamente el puntaje en el archivo `.json` y muestra el promedio general y los puntajes en las 8 categorías de ingeniería de software.	**Aprobado (Pass)** 20/oct/2026 (Tras corregir DEF-S2-02)	Rogelio Elian Loera Garay
CP-S2-05	CU-02: Estadísticas por categoría (Flujo alternativo 2.1 - Categorías sin partidas)	1. Crear un archivo de guardado nuevo. 2. Jugar únicamente en la categoría "Lógica". 3. Consultar la pantalla de Estadísticas.	"Lógica" muestra el puntaje obtenido y las 7 categorías restantes muestran `0.0 pts` con la indicación "Sin partidas registradas aún".	**Aprobado (Pass)** 20/oct/2026	Rogelio Elian Loera Garay
CP-S2-06	CU-03: Jugar Ronda Aleatoria (Flujo normal y alt. 4.1)	1. Seleccionar "Ronda Aleatoria" en el menú. 2. Resolver la serie de preguntas combinadas al azar entre las 8 categorías y dificultades.	El sistema evalúa cada turno del 1.0 al 10.0, registra 0.0 si falla y guarda el promedio al finalizar la ronda.	**Pendiente de ejecución** (Programado 27-28 oct)	Por ejecutar (Paul Rivera / Diego Alvarado)
CP-S2-07	CU-04: Copiar o borrar archivo de guardado (Flujo normal y alt. 3.1)	1. En Seleccionar Partida, elegir gestionar una ranura existente. 2. Probar "Copiar partida" a una ranura libre y "Borrar partida" confirmando el diálogo.	El archivo `.json` se clona en la ranura destino o se elimina del almacenamiento interno actualizando los botones de la interfaz.	**Pendiente de ejecución** (Programado 27-29 oct)	Por ejecutar (Diego Alvarado / Paul Rivera)

3.0 Registro de Defectos Detectados y Corregidos en la Iteración 2 (WP.15)

Defecto DEF-S2-01: El temporizador de Modo Maratón seguía corriendo en segundo plano al finalizar el tercer error

- **Caso de prueba relacionado:** `CP-S2-03` (CU-01: Iniciar Modo Maratón).
- **Detectado por:** Sebastian Del Hoyo Alvarado (en revisión de código / pruebas) | **Corregido por:** Rogelio Elian Loera Garay.
- **Fecha de detección y resolución:** 19 de octubre de 2026.
- **Severidad:** Alta (Fallo de control de flujo en fin de partida).
- **Función afectada:** Método de evaluación de turno y control de `CountDownTimer` en `ModoMaratonActivity.java`.
- **Causa raíz:** Cuando el jugador cometía el tercer error de forma manual antes de que el reloj llegara a cero, se abría el resumen de fin de maratón sin cancelar la instancia activa del `CountDownTimer`, provocando un evento adicional al agotarse los segundos.
- **Solución aplicada:** Se invocó explícitamente `temporizador.cancel()` antes de evaluar el límite de 3 errores y dentro del método `onDestroy()` de la actividad.

Defecto DEF-S2-02: Categorías nuevas de Ingeniería de Software no aparecían inicializadas en archivos JSON creados en la Sprint 1

- **Caso de prueba relacionado:** `CP-S2-04` y `CP-S2-05` (CU-02: Registrar y consultar estadísticas de progreso por categoría).
- **Detectado y corregido por:** Rogelio Elian Loera Garay (verificado en revisión por Sebastian Del Hoyo Alvarado).
- **Fecha de detección y resolución:** 20 de octubre de 2026.
- **Severidad:** Media (Compatibilidad de persistencia entre versiones).
- **Función afectada:** Deserialización de categorías en `Estadisticas.java` y `AdministradorArchivos.java`.
- **Causa raíz:** Los archivos `.json` guardados durante la primera iteración solo tenían registradas 3 categorías iniciales, por lo que al abrir la nueva pantalla de Estadísticas para las 8 categorías de Ingeniería de Software se omitían las 5 categorías restantes si no se validaban por defecto.
- **Solución aplicada:** Se actualizó el constructor y el método de carga de `Estadisticas` para asegurar que las 8 categorías oficiales (*Programación, Introducción a la ingeniería de software, Lógica, Matemáticas discretas, Ingeniería de requerimientos, Análisis y diseño orientado a objetos, Programación orientada a objetos y Análisis y diseño de algoritmos*) se inicialicen en `0.0` si no existen previamente en el archivo `.json`.

4.0 Cálculo de Métricas de Calidad y Densidad de Defectos (Corte al 20 de octubre)

Para atender el punto **3.B** y el hallazgo **E** de la auditoría de la Iteración 1, se documenta el cálculo cuantitativo de la **Densidad de Defectos** sobre los módulos construidos y probados hasta el día de hoy:

Métrica de Calidad (TSP / ISO/IEC 29110)	Valor Registrado al Corte (20 de octubre de 2026)
Casos de Uso construidos y verificados al corte	2 casos de uso completados (CU-01 Modo Maratón y CU-02 Estadísticas por categoría).
Casos de prueba ejecutados / aprobados	5 de 5 casos de prueba ejecutados aprobados (100% de éxito tras corrección) + 2 casos pendientes para CU-03 y CU-04.
Tamaño del código nuevo/modificado en Sprint 2 al corte (KLOC)	Aproximadamente **920 líneas de código efectivas (LOC) = 0.92 KLOC** (sumando `ConfigurarMaratonActivity`, `ModoMaratonActivity`, `EstadisticasActivity`, actualizaciones en `Estadisticas`, `AdministradorArchivos` y layouts XML).
Total de defectos detectados en pruebas de verificación	**2 defectos** (`DEF-S2-01` y `DEF-S2-02`).
Total de defectos corregidos antes de liberación	**2 defectos corregidos (100% resueltos)**; 0 defectos abiertos en los módulos terminados.
Densidad de Defectos por Caso de Uso	`Densidad = 2 defectos / 2 Casos de Uso = 1.00 defecto por caso de uso` (mejora respecto a la Iteración 1, donde se registraron 4 defectos en 4 casos de uso).
Densidad de Defectos por KLOC (Mil líneas de código)	`Densidad = 2 defectos / 0.92 KLOC = 2.17 defectos / KLOC`.